BIMANUAL ROBOTICS · FINAL PROJECT · JUNE 2026

Global Liquor Exchange

A two-armed robot bartender that takes a drink order and builds the cocktail end-to-end.

Rushav Dash · Tony Chen · Joyce Zhou

Bimanual SO-101 · YOLO + visual servoing + MoveIt 2 · ROS 2 Humble



THE TEAM · CONTRIBUTIONS

Each of us owns a subsystem — and presents that part today



Rushav Dash

ORCHESTRATION & CONTROL

◆

- bartender_ui orchestration + two-arm sequencing
- Wrist-camera COCO YOLO + visual-servo loop
- End-to-end pipeline integration & tuning

Presents *pipeline, challenges & evaluation; drives the live demo*



Tony Chen

PERCEPTION & MANIPULATION

◆

- Early LeRobot / ACT policy (V1)
- Overhead YOLO detection + dataset labeling
- Inverse kinematics, pose estimation & tuning

Presents *system architecture and the V1 → V2 pivot*



Joyce Zhou

INTERFACE, DATA & MOTIONS

◆

- Designed & built the bartender UI
- Recorded pour / toss poses & trajectories
- Data collection + YOLO image labeling

Presents *what the system does, and the results*

THE APPLICATION

Each cocktail is one alcohol plus one mixer, made by two dedicated arms

8 cocktails + 8 single liquids on a one-screen menu.

Right arm = alcohols, left arm = mixers.

Sequential, not simultaneous — the two arms never share the workspace, so there is no collision risk.

Live camera dots + Refresh restart any wrist detector mid-service.



The bartender UI — menu, live camera health, and single-pour controls.

SYSTEM ARCHITECTURE

Three subsystems turn a button press into a poured drink

ORCHESTRATION

bartender_ui (tkinter)

- Menu + camera health
- Owns the 2 wrist YOLOs
- Sequences the two arms

→

PERCEPTION · ML + CV

Find + align the bottle

- Overhead YOLO + depth → 3D pose
- Wrist YOLO + visual-servo loop
- Aligns in image space

→

MOTION & CONTROL

MoveIt 2 + trajectories

- Per-arm collision-aware plans
- ROS 2 action servers
- Recorded pour / toss replay

→

HARDWARE

Bimanual SO-101

- 2× 5-DoF arms + grippers
- 1× overhead RealSense D435
- 2× wrist USB cameras

Button press → detect & localize → plan & approach → grab, pour, toss → **poured drink**

Depth trick: sample the nearest 10% of depth pixels inside each YOLO box, so the table doesn't drag the reported depth back.

THE PIPELINE

Both arms run the same six phases; a full cocktail takes about 2 min 15 s

1	2	3	4	5	6
Find	**Go to**	**Align**	**Grab**	**Pour**	**Toss**
Overhead YOLO + depth pose	MoveIt plan to a standoff pose	Wrist-cam visual servo (closed loop)	Lower, level, open, enter, close, lift	Recorded joint-space trajectory	Recorded toss + release into glass
~3 s	~7 s	~10 s	~13 s	~18 s	~16 s

One arm, 6 phases ≈ 1 min 7 s. Arms run sequentially (right then left), so a full two-ingredient cocktail — all 12 phases — takes ≈ 2 min 15 s.

LIVE DEMO

Making a cocktail from the menu, end to end



Watch for: overhead detection → standoff approach → wrist-camera alignment → grab → pour → toss, then the second arm.

Demo layout: 4 bottles per shelf (2 top, 2 bottom) — keeps both the easy and the hard, occluded cases.

Backup video (if hardware fails): final run linked in the repo README.

CHALLENGES · HOW WE IMPROVED THE SYSTEM

An unstable LeRobot + Rosetta integration drove our pivot to a modular classical pipeline

V1 — End-to-end imitation learning

- Teleop demos → trained an ACT imitation policy (LeRobot).
- A real proof of concept — it did grab and pour.
- But it ran through Rosetta, which applied a constant offset to the SO-101 shoulder joint.
- **That made the LeRobot + Rosetta integration unstable and unpredictable.**
- A black box on top of it — hard to inspect a bad run.

→

V2 — Modular classical + ML pipeline

- Every phase is explicit, testable, tunable.
- Perception decoupled from control.
- **Failures attributable to a specific stage.**
- Enables the failure-mode analysis below.
- Traded some autonomy for safety + repeatability.

The learned policy remains in the repo history as a working proof of concept.

ENGINEERING CHALLENGES OVERCOME

Hardest problems: 5-DoF grasp geometry, camera plumbing, safe two-arm coordination

5-DoF grasp geometry	Level the wrist to the ground (not the forearm) via TF pitch; approach along the gripper axis so it doesn't slide and tip the bottle; lift in joint space (cartesian lifts loop).
Wrist-camera plumbing	Two identical USB cams collide on serial — pin each by physical USB port (/dev/v4l/by-path). MJPEG segfaulted, so we stream raw YUYV → RGB.
Detection robustness	COCO labels bottles as 'bottle' OR 'vase' by material — accept both, lock onto the largest (closest) box; back off the standoff for a cleaner view.
Gravity sag + resilience	Centering loop holds non-pan joints fixed and pre-compensates droop per arm; pour / toss retries a failed pose once and continues.
Safe two-arm coordination	Simultaneous planning through the shared workspace fails and is unsafe — we run the arms sequentially: collision-free and reliable.

QUANTITATIVE EVALUATION · EXPERIMENT DESIGN

480 trials, a strict binary success test, varying shelf position

Success (binary): grab the correct bottle, lift it clear, pour, and release it in the drop zone — without dropping it elsewhere or knocking over a neighbor.

Failure / termination: not grasped; dropped outside zone; neighbor knocked over; MoveIt fails after retries; or 2-min timeout.

Start state: 8 bottles upright in assigned cells, both arms home, glass at drop zone.

Reset: re-stand poured + any knocked bottles, arms to home, replace glass.

VARIED PARAMETER

Shelf position

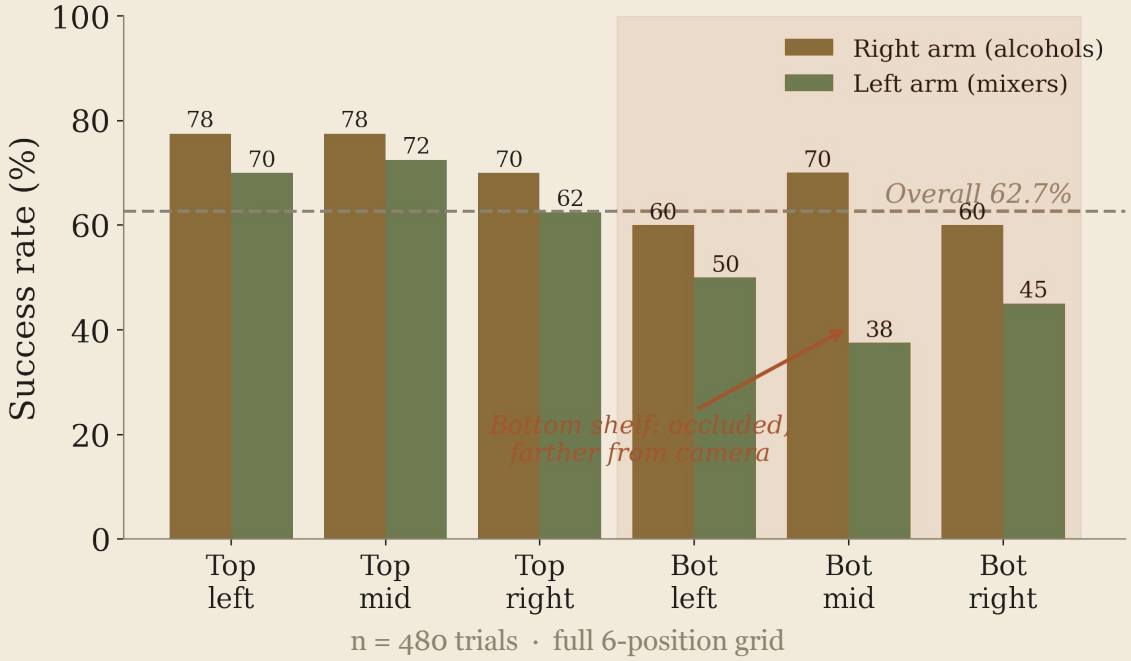
Top / bottom row × left / mid / right = 6 positions per shelf. Lower cells are occluded by the shelf above and sit farther from the overhead camera.

480	12	10
total trials	shelf positions	trials per cell

Each (position, drink) cell run 10× — ≥ 30 trials per state, 480 total.

RESULTS · SUCCESS RATE

Overall success is 62.7%; top-shelf bottles beat occluded bottom-shelf ones



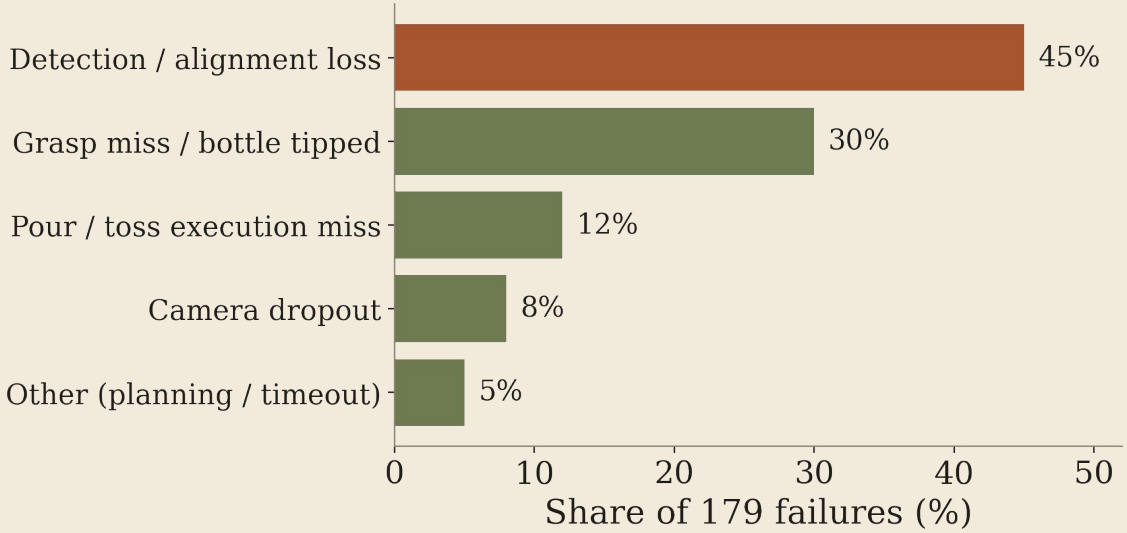
n = 480 trials · full 6-position grid

WHAT IT SHOWS

- **Top shelf 71.7% vs bottom 53.8%**
- Alcohols (right, 69.2%) beat mixers (left, 56.3%).
- Best liquid: vodka 73.3%; worst: coke 48.3%.
- **Occlusion + camera distance drive the gap.**

RESULTS · FAILURES & TIMING

Detection / alignment loss on the bottom shelf is the dominant failure mode



TIMING · BEST CASE

68 s

one arm / single liquid (std ≈ 8 s)

2 min 15 s

full two-ingredient cocktail (12 phases)

Mitigations: per-shelf lower / forward offsets; retry-then-continue on pour / toss; GUI Refresh recovers camera dropouts.

CONCLUSIONS

1. It works end-to-end. A bimanual robot makes a full two-ingredient cocktail from a menu in ~2:15, 62.7% of the time over 480 trials.

2. Modular beats black-box here. Pivoting from a learned policy to an explicit pipeline bought us safety, repeatability, and debuggable failures.

3. Occlusion is the bottleneck. The top/bottom-shelf gap (71.7% vs 53.8%) and most failures trace to one cause: occluded, distant bottom-shelf views.

4. Designed to be safe. Sequential arms, a gripper-axis approach, and retry-then-continue keep it safe and recoverable.



Repo: github.com/tonyechen/drunky_ros *Next: close the occlusion gap (second viewpoint / active sensing).*

TOOLS, FRAMEWORKS & REFERENCES

Built on open-source robotics and vision

- Ultralytics YOLO — object detection (overhead 8-shooter detector + COCO wrist model). github.com/ultralytics/ultralytics (AGPL-3.0).
- yolo_ros — ROS 2 YOLO integration. github.com/mgonzs13/yolo_ros (MIT).
- MoveIt 2 — motion planning. moveit.ai (BSD-3-Clause).
- pymoveit2 — Python MoveIt 2 interface. github.com/AndrejOrsula/pymoveit2 (BSD-3-Clause).
- LeRobot — ACT imitation-learning policy (V1 proof of concept). github.com/huggingface/lerobot (Apache-2.0).
- ROS 2 Humble, realsense2_camera, usb_cam — middleware, RealSense D435, USB wrist cameras.
- SO-101 follower arms — bimanual 5-DoF manipulator platform.

Project license: AGPL-3.0 (required by Ultralytics YOLO). Repo: github.com/tonyechen/drunky_ros